HyreKit

Сопроводительная документация · хакатон ФСП 2026, «Команда 1»

10.10.2026

Содержание

- Введение
- 1. Продукт
- 2. Как работает
- 3. Методика оценки
- 4. Архитектура
- 5. Запуск и демо
- 6. Технологии и сервисы
- 7. Команда и процесс

Введение

Документация для жюри хакатона ФСП и для любого человека вне команды. Её задача — чтобы сторонний читатель понял продукт, механику тестирования и подбора и воспроизвёл запуск без пояснений команды (ТЗ, п. 3.4). Трек — «Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника ФСП», команда «Команда 1».

Версия продукта — **0.1.0** от 10.10.2026. Описано только то, что есть в коде `main`; чего нет, названо прямо: «не входит в MVP».

Ссылки

Что	Адрес
Прототип	https://hyrekit.ru — площадка в Yandex Cloud готовится к выкатке; до неё прототип запускается локально за одну команду (раздел 5). Демо-входы: `candidate@hyrekit.ru` и `employer@hyrekit.ru`, пароль `HyreKit2026`
Описание API (Swagger)	https://hyrekit.ru/docs, локально — http://localhost:8000/docs
Одним файлом	HyreKit_Documentation.docx и .pdf — собраны из этих разделов

Что	Адрес
	командой `python scripts/build_docs.py`
Спецификация OpenAPI	docs/openapi.json — выгружается `scripts/export_openapi.py`, CI проверяет, что файл совпадает с кодом
Репозиторий	https://github.com/Fighter90/hyrekit_fsp — код открыт, без обфускации, зависимости закреплены в `requirements.txt` и `frontend/package-lock.json`

Разделы

№	Раздел	Что внутри	Пункт ТЗ
1	Продукт	Проблема, обратная механика найма, роли, границы MVP	1, 2, 2.1
2	Как работает	Сквозной сценарий: регистрация → опрос → тест → категория → подборка → приглашение	2.1, 3.3
3	Методика оценки	Тестирование, устойчивость к утечке, категоризация, рейтинг, подбор, валидация и результаты	2, 3.4
4	Архитектура	Функциональная и компонентная архитектура, данные, API, интеграция с реестром ФСП	3.4, 3.5
5	Запуск и демо	Сборка, развёртывание, локальный	3.3, 3.4

№	Раздел	Что внутри	Пункт ТЗ
		запуск, демо-сценарий для жюри	
6	Технологии и сервисы	Библиотеки и компоненты с версиями, лицензии, GigaChat	3.2, 3.4, 3.5
7	Команда и процесс	Состав, роли, SDD, задачи Linear, качество	5

Как читать за 10 минут

1. Раздел 1, абзацы «Проблема» и «Обратная механика» — 2 минуты.
2. Раздел 3, «Почему тест не утекает» и «Результаты валидации» — 5 минут.
3. Раздел 5, «Демо за 5 минут» — пройти руками.

Полные цифры валидации пересчитываются командой `PYTHONPATH=src python scripts/validate.py` и лежат в docs/validation/results.md.

1. Продукт

Проблема

Универсальные площадки поиска работы не решают задачу ни одной из сторон. Работодатель вручную разбирает сотни нерелевантных откликов силами дорогих инженеров. Кандидат откликается вслепую и не получает ответа. В ИТ это усугубляется тем, что резюме фиксирует стек и места работы, но не подтверждает умение: отбор сводится к совпадению ключевых слов (ТЗ, п. 1).

Обратная механика

HyreKit переворачивает привычный путь. Инициатива у работодателя:

1. Кандидат один раз проходит обязательный путь: опрос по отрасли и специализации, выбор грейда, тест на этот грейд.
2. Тест присваивает **категорию** — связку специализации и грейда, например «Backend · Middle». Самоописание из резюме на категорию не влияет.
3. Внутри категории кандидаты ранжируются по силе подтверждённого профиля: результат теста, достижения ФСП, свежесть активности.

4. Работодатель описывает потребность и получает подборку с объяснением, почему каждый кандидат в ней. Он сам выбирает человека и отправляет **приглашение с вилкой зарплаты** — кандидат видит условия до начала общения.
5. Контакты кандидата скрыты, пока он не принял приглашение или не откликнулся сам.

Классический отклик на вакансию тоже есть, но как дополнение: кандидат, которого ещё не пригласили, может проявить инициативу сам (ТЗ, п. 2.1).

Роли

Роль	Что делает
Кандидат	Заполняет профиль вручную или загрузкой PDF, проходит опрос и тест, получает категорию, привязывает ФСП ID, управляет приватностью, принимает или отклоняет приглашения, откликается на вакансии, решает короткие задания работодателей
Работодатель	Ведёт профиль компании, описывает потребности (вакансии), получает подборку с объяснением, ищет по банку кандидатов, отправляет приглашения, видит тест под вакансию, публикует вакансии и разбирает отклики, ставит короткие задания

Роль выбирается при регистрации. Эндпоинты кабинета кандидата недоступны работодателю и наоборот; каждый видит только свои записи.

Что такое категория и грейд

Грейд	Типичный опыт	Уровень заданий
Intern	без опыта	1
Junior	до 1–2 лет	2
Middle	2–4 года	3
Senior	4–7 лет	4
Lead	7+ лет	5

Направления MVP: Backend, Frontend, Data Science / ML, DevOps / SRE, QA, информационная безопасность. Справочник лежит в `src/app/catalog/data.py`, отдаётся эндпоинтом `GET /api/v1/catalog`.

Границы MVP

Входит:

- оба кабинета со всеми обязательными пунктами ТЗ 2.1 и правилами видимости 2.2;
- регистрация по почте с подтверждением адреса;
- механика тестирования с параметрическими заданиями и оценкой способности;
- подборка с объяснимостью, поиск с фильтрами, приглашения со статусами;
- дополнительный функционал: публичные вакансии и отклики, регулярные короткие задания, адаптивная вёрстка;
- эмулятор реестра ФСП с корректной обработкой кандидата без истории;
- валидация методики на синтетических данных с воспроизводимым скриптом.

Не входит в MVP (концепция и архитектурный подход — в разделе 4):

- реальная интеграция с реестром ФСП и ФСП ID на Keycloak;
- защита от фиктивных вакансий и недобросовестных работодателей, верификация компаний;
- интеграция с ATS работодателей.

Почему это работает для обеих сторон

Сторона	Было	Стало
Работодатель	Сотни откликов, ручная фильтрация по ключевым словам	Подборка из категории, подтверждённой тестом, с причинами попадания и недостающим стеком
Кандидат	Отклик вслепую, зарплата неизвестна	Приглашение приходит само, вилка зарплаты видна сразу, контакты раскрываются только с согласия
Участник ФСП	Спортивные результаты не видны на рынке труда	Достижения автоматически попадают в профиль и поднимают кандидата внутри категории

2. Как работает

Сквозной сценарий из ТЗ 3.3: регистрация соискателя → опрос и тест → категория → работодатель находит категорию → приглашение → контакт. Плюс классический отклик на вакансию.

Кандидат

Шаг	Экран	Что происходит в системе	API
1	Регистрация	Роль «кандидат», письмо со ссылкой подтверждения адреса	`POST /auth/register`, `GET /auth/verify/{token}`
2	Профиль	Ручное заполнение или загрузка PDF/DOCX. Текст разбирается правилами и, если подключён GigaChat, моделью: ФИО, контакты, стек, роли, стаж, софт-скиллы	`PUT /candidate/profile`, `POST /candidate/profile/resume`
3	Опрос	Отрасль, специализация, стаж, стек, предполагаемый грейд	`POST /candidate/assessment/survey`
4	Тест	10 заданий по 5 темам специализации, 20 минут, ответы сохраняются по одному, попытку можно продолжить	`POST /candidate/assessment/start`, `.../answer`, `.../finish`

Шаг	Экран	Что происходит в системе	API
5	Результат	Оценка способности θ, балл 0–100, разбор по темам. Сдал — грейд и категория присвоены. Не сдал — грейд не меняется, система предлагает тест уровнем ниже. Сдал уверенно — предлагает уровень выше	`GET /candidate/assessment/history`
6	Настройки	Согласие на обработку и публикацию (152-ФЗ), что видно работодателю, ФСП ID	`PUT /candidate/settings`
7	Приглашения	Компания, описание предложения, вилка зарплаты, способ связи. Принять — работодатель видит контакты; отклонить — с причиной	`GET /candidate/invitations`, `.../accept`, `.../decline`
8	Вакансии	Список опубликованных вакансий, отклик с сообщением, статусы откликов	`GET /candidate/vacancies`, `POST .../apply`
9	Задания	Короткие задачи от работодателей: ответ оценивается	`GET /candidate/challenges`, `POST .../submit`

Шаг	Экран	Что происходит в системе	API
		GigaChat по рубрике или эвристикой без модели, профиль становится «свежее»	
10	PDF-профиль	Стандартизированный профиль в фирменной гарнитуре	`GET /candidate/profile/pdf`

Работодатель видит кандидата, только когда у него есть категория **и** согласие на публикацию.

Работодатель

Шаг	Экран	Что происходит в системе	API
1	Регистрация	Роль «работодатель»	`POST /auth/register`
2	Компания	Название, направление, описание, контакты	`GET/PUT /employer/company`
3	Потребность	Кого ищем, чем занимается команда, специализация, диапазон грейдов, стек, вилка зарплаты (обязательна), формат работы	`POST /employer/vacancies`
4	Подборка	Категории с числом кандидатов, список по убыванию соответствия, разложение балла и причины: «Категория	`GET /employer/vacancies/{id}/matches`

Шаг	Экран	Что происходит в системе	API
		Backend · Middle входит в диапазон», «Стек: 4 из 5», «ФСП: 2 соревнования, лучшее место — 1». Фильтры по грейду, стеку и ФСП уточняют список, не сбрасывая подборку	
5	Тест под вакансию	Темы теста, выведенные из стека вакансии, и два варианта с разными заданиями и одинаковой сложностью	`GET /employer/vacancies/{id}/assessment`
6	Банк кандидатов	Поиск по всей базе: специализация, грейд, категория, стек, наличие достижений ФСП, текст	`GET /employer/candidates`
7	Приглашение	Описание, вилка «от–до» в рублях, способ связи; без обязательной привязки к вакансии. Повторное приглашение, пока первое не закрыто, запрещено (409)	`POST /employer/invitations`
8	Статусы	отправлено → просмотрено →	`GET /employer/invita`

Шаг	Экран	Что происходит в системе	API
		принято / отклонено; после принятия видны ФИО и контакты	`tions`
9	Отклики	Входящие отклики на вакансии, смена статуса; откликнувшийся кандидат раскрывает контакты сам	`GET /employer/applications`, `POST .../status`
10	Задания	Короткие задачи для кандидатов направления и ответы на них с оценкой	`GET/POST /employer/challenges`

Правила видимости (ТЗ 2.2)

Правило	Где в коде
Зарплата обязательна в вакансии и приглашении, `from ≤ to`	схемы `VacancyIn` и `InvitationIn` в `src/app/employers/router.py`
Контакты скрыты до принятия приглашения или отклика	`candidates.service.to_card`, `hiring.service.unlocked_profiles`
Без согласия и категории профиль не виден	`candidates.service.is_visible`
Скрытое ФИО заменяется псевдонимом «Кандидат #A1B2»	`candidates.service.public_alias`
Кандидат видит только свои приглашения и отклики	фильтр по `profile_id` во всех запросах `/candidate/*`
Смена грейда не чаще раза в 90 дней	`assessment.service.eligibility`

3. Методика оценки

Раздел отвечает на три вопроса ТЗ 3.4: как устроено тестирование и почему оно устойчиво к распространению заданий, как работают категоризация и подбор, как команда проверяла решение и что

получилось. Все числа ниже пересчитываются командой `PYTHONPATH=src python scripts/validate.py`; полный отчёт — docs/validation/results.md.

Почему не единый набор и не задания от нейросети

ТЗ отвергает оба очевидных пути (п. 2):

Подход	Проблема
Единый набор заданий	Ответы расходятся по рынку за недели. В нашей симуляции утечка единственного варианта даёт слабому кандидату 100 % прохождения
Уникальные задания от LLM под каждого	Сложность и объективность между кандидатами не гарантированы: модель может сгенерировать задачу проще или с ошибкой в ответе, и сравнить кандидатов уже нельзя

HyreKit берёт лучшее от обоих: задания уникальны, как у генерации, но собираются из **откалиброванных генераторов**, а ответ вычисляет код, а не модель.

Банк параметрических заданий

Банк (`src/app/assessment/bank.py`) хранит не вопросы, а **36 семейств** — генераторов по 14 темам: алгоритмы, Python, JavaScript, веб, HTTP, SQL, архитектура, ML, статистика, Linux, сети, контейнеры, тестирование, безопасность.

Семейство по зерну (`seed`) и уровню сложности (`level` 1–5) порождает конкретное задание: другие числа, данные, имена, код, порядок вариантов. Правильный ответ вычисляется тем же кодом. Пример — семейство `http_backoff`:

> Каждый запрос к API завершается таймаутом через 500 мс. Клиент делает первую попытку и до 3 повторов с экспоненциальной паузой: первая пауза 330 мс, каждая следующая в 2 раза больше. Через сколько миллисекунд от начала клиент сдастся?

У следующего кандидата будут другой таймаут, другая пауза, другое число повторов; на уровне 4 добавится потолок паузы, на уровне 3 — множитель 3. Выученное «правильное число» не переносится, перенести можно только понимание.

Форматы ответов: один вариант, несколько вариантов, число. Задачи с кодом — вывод программы, сложность алгоритма, результат SQL-запроса, ширина flex-элемента, права `chmod`, широковещательный адрес подсети.

Два класса семейств

Класс	Сколько	Что проверяет	Пространство вариантов
Генеративные	23	Навык: вычислить, предсказать вывод, разобрать данные	Практически бесконечное
Концептуальные	13	Знание понятия: идемпотентность методов, паттерн, уязвимость в фрагменте	Конечное, формулировки повторяются

Концептуальные задания быстро «выучиваются» рынком, поэтому в плане попытки первое задание каждой темы всегда генеративное, а концептуальных не больше одного на попытку (`engine.MAX_CONCEPTUAL`). Это решение принято по результатам атаки утечкой, см. ниже.

План попытки и равная сложность

Попытку собирает `engine.build_blueprint(specialization, grade)`:

1. Берутся 5 тем специализации (у Backend — алгоритмы, Python, SQL, HTTP, архитектура).
2. На каждую тему 2 задания: уровень грейда (`order + 1`) и соседний уровень — на чётных темах выше, на нечётных ниже.
3. Для каждого слота случайно выбирается семейство, допустимое на этом уровне (у семейства есть минимальный осмысленный уровень), и случайное зерно.
4. Задания перемешиваются; на попытку 10 заданий и 20 минут.

Профиль сложности — мультимножество уровней — определяется только грейдом. У всех кандидатов, тестируемых на Middle, он один и тот же: `[2, 2, 3, 3, 3, 3, 3, 4, 4, 4]`. Валидация собирает по 1 200 случайных вариантов на каждый грейд и проверяет это по уровням **сгенерированных** заданий: различных профилей — ровно один.

План попытки (семейства, уровни, зёрна) хранится в базе, поэтому попытку можно продолжить после перезагрузки, а результат — перепроверить.

Оценка способности и решение по грейду

Ответы переводятся в оценку способности θ по модели Раша (однопараметрическая IRT): вероятность верного ответа на задание сложности b равна `σ(θ − b)`, где `b = level − 3`. Оценка — MAP с априорным N(0, 1), метод Ньютона (`engine.estimate_ability`).

Пороги грейдов на шкале θ: `cut(g) = order(g) − 2.5`, то есть Intern −2.5, Junior −1.5, Middle −0.5, Senior 0.5, Lead 1.5. Коридор каждого грейда — один логит.

Решение (`engine.decide`) следует ТЗ:

Исход	Что происходит
θ ≥ порога тестируемого грейда	Грейд присвоен, категория `специализация:грейд` видна работодателям
θ ≥ порога следующего грейда + 0.35	Плюс подсказка «попробуйте уровень выше»
θ ниже порога	Грейд **не понижается**, предлагается тест уровнем ниже
У кандидата уже есть более высокий грейд	Он сохраняется — результат слабее не отнимает подтверждённое

Смена грейда — не чаще раза в 90 дней (`GRADE_COOLDOWN_DAYS`). Пока грейда нет, можно пробовать любой уровень сколько угодно раз: так кандидат, не сдавший заявленный уровень, сразу сдаёт уровень ниже.

Помимо θ кандидат видит балл 0–100 (доля верных, взвешенная по сложности) и разбор по темам.

Категоризация и рейтинг внутри категории

Категория — связка специализации и подтверждённого грейда: «Backend · Middle». Она задаётся **только тестом**; самоописание из резюме и анкеты на неё не влияет (ТЗ 2.1, требования к подбору).

Внутри категории кандидаты ранжируются по силе подтверждённого профиля (`candidates.service.rating`):

Компонент	Вес	Как считается
Результат теста	65 %	`100 · σ(1.5 · (θ − cut(грейд)))` — насколько уверенно кандидат прошёл порог своего грейда
Достижения ФСП	25 %	Балл 0–100: уровень

Компонент	Вес	Как считается
		соревнования × место × давность; дисциплина, профильная для специализации, весит вдвое больше
Свежесть	10 %	100 в течение 30 дней после активности, затем линейно до 0 к 180 дням. Активность — тест, правка профиля, ответ на короткое задание

Кандидат без истории ФСП получает 0 по этому компоненту и ранжируется по тесту и свежести — профиль обрабатывается корректно, как требует ТЗ.

Механика подбора

Работодатель описывает потребность: специализация, диапазон грейдов, стек, описание задач и команды, вилка зарплаты. Подборка (`matching.service.match_vacancy`) строится так:

1. **Кандидатный пул** — видимые профили нужной специализации: есть категория и согласие на публикацию.
2. **Требуемый стек** — стек вакансии плюс технологии, найденные в тексте описания (синонимы нормализуются: `k8s` → Kubernetes, `postgres` → PostgreSQL).
3. **Балл соответствия 0–100**:

Компонент	Вес	Как считается
Категория	30 %	Грейд в диапазоне — 100; на ступень выше — 70 (запас квалификации); на ступень ниже — 55 (потенциал роста); дальше — 0
Стек	30 %	Доля требуемых технологий, которые есть у кандидата
Уровень по тесту	25 %	Насколько измеренная θ попадает в коридор

Компонент	Вес	Как считается
		вакансии: внутри — 100, снаружи гауссово затухание с σ = 0.5 логита
ФСП	10 %	Балл достижений
Свежесть	5 %	Как в рейтинге

4. **Объяснимость**: к каждому кандидату — причины («Категория Backend · Middle входит в требуемый диапазон», «Стек: 4 из 5 (Python, FastAPI, PostgreSQL, Kafka)», «Тест на грейд: 78/100», «ФСП: 2 соревнования, лучшее место — 1, профильных — 1») и список недостающих технологий.
5. **Уточнение без потери подборки**: фильтры по грейду, обязательному стеку и наличию ФСП применяются к уже построенному ранжированию; в интерфейсе прежний список остаётся на экране, пока грузится уточнённый, и «Сбросить фильтры» возвращает исходный.

Компонент «Уровень по тесту» сравнивает θ с коридором вакансии, а не с порогом собственного грейда кандидата. Так сильный кандидат, которому тест присвоил Middle, но чья θ на уровне Senior, не вытесняет подходящих на вакансию Junior–Middle. Это решение тоже пришло из валидации: с прежней формулой точность подборки для фронтенд-вакансии была 0.73, с новой — 0.89.

Тест под вакансию

ТЗ просит показать на нескольких описаниях вакансий, как система формирует задания (п. 2). Эндпоинт `GET /employer/vacancies/{id}/assessment` и вкладка «Тест под вакансию» в кабинете работодателя:

1. Каждая технология стека вакансии подтверждает тему теста (`catalog.data.TECH_TOPICS`): Python → Python, PostgreSQL → SQL, Kafka → архитектура, Kubernetes → контейнеры.
2. Темы сортируются по числу подтверждающих технологий; недостающие до пяти добираются базовыми темами специализации. Каждая тема показана с причиной: «Стек вакансии: Python, FastAPI».
3. Строятся два варианта с разными заданиями. Их профили сложности совпадают между собой и со стандартным тестом категории — это проверяет тест `test_vacancy_test_topics_follow_stack_and_keep_difficulty`.

Регулярные короткие задания

Работодатель публикует задачу для направления («Как сделать платёж идемпотентным?»). Кандидат отвечает текстом. Ответ оценивает GigaChat по рубрике (понимание задачи, корректность, конкретика, учёт рисков); без ключа модели — эвристика по полноте и ключевым словам задачи. Ответ обновляет свежесть профиля и виден работодателю вместе с оценкой.

Процедура валидации

Готовых датасетов и эталонной разметки в хакатоне нет (ТЗ, п. 2), поэтому команда валидирует методику на **синтетических кандидатах с известной истинной способностью**. Модельный кандидат с θ отвечает на задание сложности b верно с вероятностью $\sigma(\theta - b)$. Истинный грейд — наивысший, чей порог не выше θ. Это позволяет измерить то, что на реальных людях потребовало бы экспертной разметки: совпадает ли присвоенный грейд с истинным.

Что проверяется и почему:

Проверка	Критерий ТЗ	Метод
Равная сложность вариантов	Сопоставимая сложность между кандидатами	1 200 вариантов на грейд, считаются различные профили сложности
Устойчивость к утечке	Устойчивость к распространению заданий	200 слитых попыток с ответами на направление; доля дословно совпавших заданий; атака Junior со шпаргалкой
Надёжность	Стабильный грейд при повторном прохождении	400 кандидатов проходят тест дважды независимо; корреляция θ, совпадение решения
Дискриминативность	Разделение слабых и сильных	1 500 кандидатов вокруг грейда, точечно-бисериальная корреляция ответа с θ по каждому семейству
Точность категоризации	Соответствие эталонной разметке	600 кандидатов, 40 % завышают заявленный грейд на ступень; присвоенный грейд

Проверка	Критерий ТЗ	Метод
		против истинного
Качество подбора	Доля релевантных в топе на парах «вакансия — кандидат»	3 демо-вакансии × 20 пулов по 150 кандидатов; precision@10 и nDCG@10 против двух базовых подходов

Результаты валидации

Равная сложность. У каждого грейда ровно один профиль сложности на 1 200 вариантов.

Утечка. По каждому из 6 направлений слито 200 попыток Middle с ответами:

Показатель	Значение
Раскрыто генеративных заданий	3.0 %
Раскрыто концептуальных заданий	52.0 %
Раскрытых заданий в попытке из 10, в среднем	0.8
Junior сдаёт Middle честно	27.9 %
Junior сдаёт Middle со шпаргалкой из 200 попыток	43.1 %
Junior сдаёт статичный тест, у которого утёк вариант	100 %

До ограничения концептуальных заданий шпаргалка поднимала долю сдавших до 71 %. Остаточный выигрыш даёт в основном единственное концептуальное задание; его можно убрать совсем (`MAX_CONCEPTUAL = 0`, тогда 33 %), но команда оставила одно ради охвата понятийных знаний.

Надёжность. Корреляция θ между двумя независимыми попытками — 0.69, решение «сдал / не сдал» совпадает в 70 % случаев, стандартная ошибка θ — 0.49 логита при ширине коридора грейда в 1 логит. Для теста из 10 заданий это ожидаемый уровень; точность растёт с длиной теста (см. «Ограничения»).

Дискриминативность. Медиана корреляции ответа с θ по 32 семействам, попавшим в выборку, — 0.27, минимум — 0.16 (порог пригодности 0.2). Ответы смоделированы по модели Раша, поэтому таблица проверяет конвейер анализа; на пилоте с реальными людьми тот же расчёт покажет семейства для переработки.

Категоризация (600 кандидатов):

Метрика	Значение
Заявленный кандидатом грейд совпадает с истинным	63.0 %
Присвоенный по тесту грейд совпадает с истинным	**70.5 %**
Ошибка не больше одной ступени	99.5 %
Грейд завышен	6.2 %

Тест не только точнее самооценки, но и почти не завышает: работодатель получает «Middle», который не окажется Junior. Ошибки — в основном занижение на ступень у кандидатов возле нижней границы коридора; такой кандидат может пересдать через 90 дней.

Подбор (precision@10 / nDCG@10, среднее по 20 пулам):

Вакансия	HyreKit	По заявленному грейду и стеку	Только по стеку
Python backend, Middle–Senior	**0.87** / 0.86	0.80 / 0.79	0.59 / 0.58
Frontend React, Junior–Middle	0.89 / 0.89	**0.90** / 0.89	0.59 / 0.60
DevOps Kubernetes, Middle–Lead	**0.96** / 0.95	0.85 / 0.86	0.70 / 0.70

Подбор по стеку, как на универсальных площадках, даёт 0.59–0.70. Подтверждённая категория поднимает долю релевантных в топ-10 до 0.87–0.96. На вакансии Junior–Middle подбор по заявленному грейду сопоставим с нашим: в узком нижнем диапазоне завышение самооценки на ступень реже выводит кандидата за пределы вакансии.

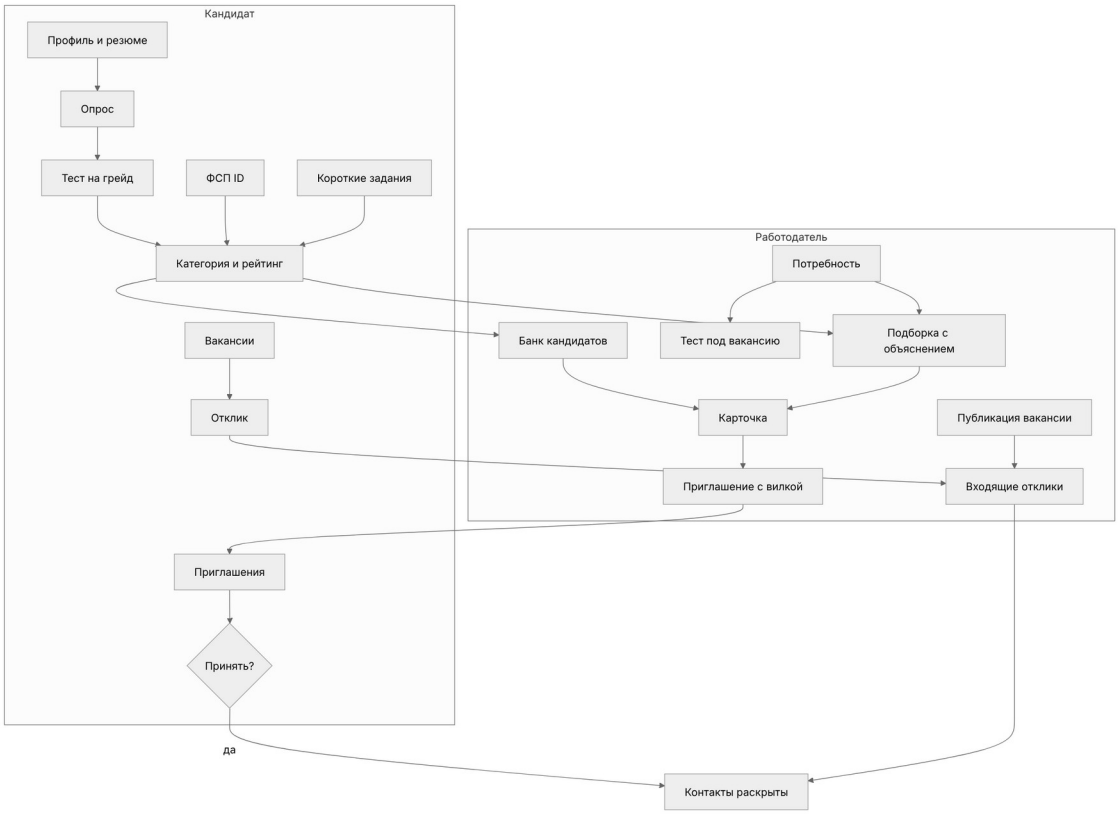
Ограничения и что дальше

- Точность грейда ограничена длиной теста: 10 заданий дают ошибку θ около половины коридора. Следующий шаг — адаптивный подбор заданий (CAT) с остановкой по точности: тот же банк, но задание выбирается по текущей оценке θ.
- Сложности семейств заданы экспертно уровнями 1–5. На пилоте их нужно откалибровать по реальным ответам и следить за дрейфом решаемости: если доля верных у семейства растёт без роста θ, его ответы утекли.
- Синтетическая валидация проверяет механику, но не содержательную валидность заданий. Её дадут экспертная оценка

выборки реальных кандидатов и сравнение с решениями нанимающих менеджеров.

4. Архитектура

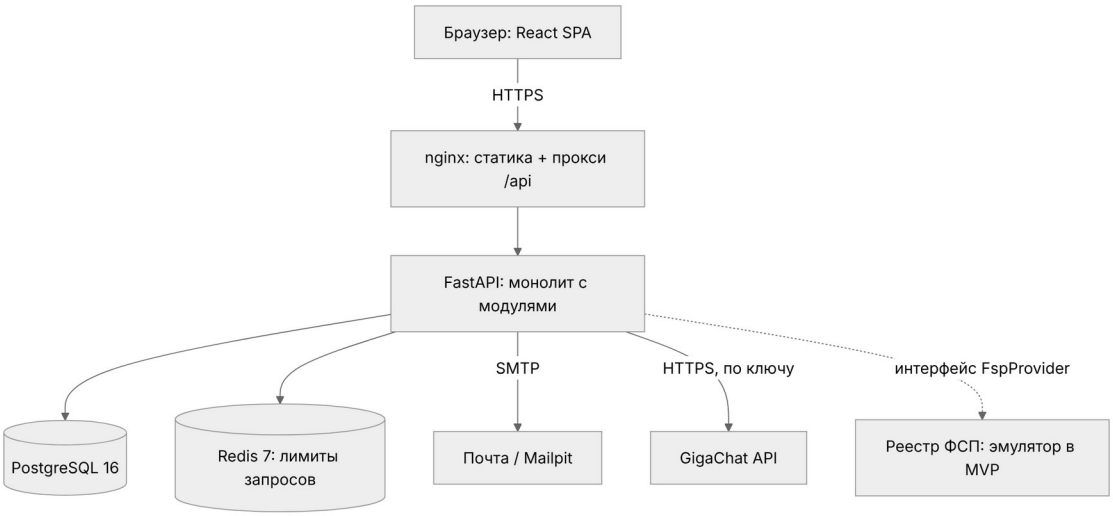
Функциональная архитектура



Функциональный блок	Модуль бэкенда	Экраны
Регистрация, вход, роли, подтверждение почты	`auth`, `core.mailer`, `core.roles`	`/auth`, `/verify/:token`
Справочник специализаций, грейдов, тем	`catalog`	опрос, фильтры
Профиль кандидата, разбор резюме, PDF-профиль, приватность	`candidates`, `ml.parser`, `llm`	профиль, настройки
Тестирование: банк, план, оценка, решение	`assessment`	тестирование
Достижения ФСП	`fsp`	настройки, карточка кандидата

Функциональный блок	Модуль бэкенда	Экраны
Подбор и поиск	`matching`	подборка, банк кандидатов
Компании и вакансии	`employers`	компания, потребности
Приглашения и отклики	`hiring`	приглашения, вакансии
Короткие задания	`challenges`	челленджи в обоих кабинетах

Компонентная архитектура

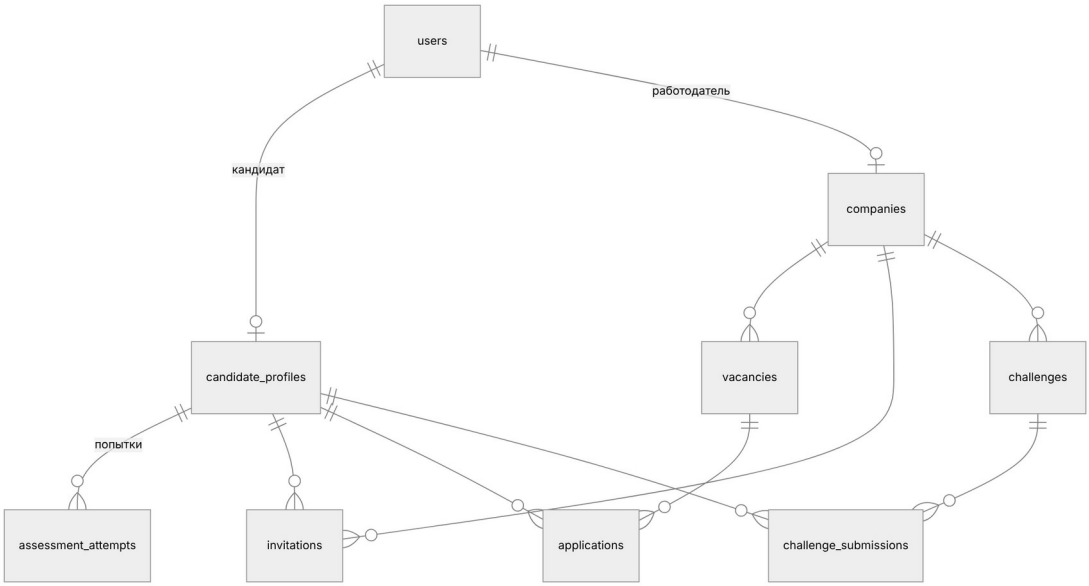


Монолит с явным разделением слоёв и слабо связанными модулями — вариант, прямо допущенный ТЗ 3.5. Каждый модуль бэкенда содержит `models.py` (SQLAlchemy), `service.py` (бизнес-логика без HTTP) и `router.py` (FastAPI). Модули общаются через сервисные функции, а не через роутеры. Методика (`assessment.bank`, `assessment.engine`, `matching.service`) — чистые функции без доступа к базе, поэтому их проверяет и скрипт валидации, и юнит-тесты.

Компонент	Технология	Роль
Фронтенд	React 19, TypeScript, Vite	Оба кабинета и лендинг, ленивые маршруты по ролям
API	FastAPI, Pydantic v2, async SQLAlchemy	Бизнес-логика, OpenAPI
Хранилище	PostgreSQL 16 (в разработке — SQLite)	Пользователи, профили, попытки, вакансии, приглашения

Компонент	Технология	Роль
Кэш и лимиты	Redis 7	Ограничение частоты входа и регистрации
LLM	GigaChat	Разбор резюме, оценка ответов на короткие задания. Без ключа работают правила
Почта	SMTP; Mailpit в docker-compose	Подтверждение адреса

Модель данных



Подробно по полям — docs/schema.md. Схема создаётся миграцией `alembic/versions/0001_initial_schema.py`.

Описание API

Все методы описаны спецификацией OpenAPI, которую генерирует FastAPI: Swagger UI — `/docs`, ReDoc — `/redoc`, JSON — `/openapi.json`, копия в репозитории — docs/openapi.json (у каждого метода есть summary, схемы запросов и ответов). Контракт для фронтенда с типами и кодами ошибок — docs/spec/api.md.

Группа	Префикс	Основные методы
Авторизация	`/api/v1/auth`	`register`, `login`, `refresh`, `verify/{token}`, `resend-verification`, `me`
Справочник	`/api/v1/catalog`	`GET` — специализации, грейды, отрасли,

Группа	Префикс	Основные методы
		форматы работы
Кандидат	`/api/v1/candidate`	профиль, резюме, PDF, настройки, ФСП, опрос, тест, приглашения, вакансии, отклики, задания
Работодатель	`/api/v1/employer`	компания, вакансии, подборка, тест под вакансию, банк кандидатов, категории, приглашения, отклики, задания
Служебное	`/health`	проверка живости

Ошибки возвращаются единым форматом `{error_code, message, detail}`; коды: 400 — валидация, 401 — нет токена, 403 — чужая роль или ресурс, 404 — нет объекта, 409 — конфликт состояния, 422 — ошибка схемы, 429 — превышен лимит.

Безопасность и 152-ФЗ

Требование ТЗ 3.5	Реализация
Согласие на обработку и публикацию	Флаг согласия с датой; без него профиль не попадает к работодателям. Текст согласия — страница `/privacy`
Разграничение прав по ролям	Зависимости `require_candidate` и `require_employer`; кандидат видит только свои записи, работодатель — только свою компанию
Защита контактов	Контакты и скрытое ФИО раскрываются только после принятия приглашения или отклика
Хранение паролей и токенов	bcrypt; JWT access 30 минут и refresh 30 дней; секрет только из окружения, слабый секрет в production останавливает запуск
Защита от перебора	Лимиты: регистрация 3 в минуту, вход 5 в минуту с адреса
Минимизация	Работодатель видит псевдоним «Кандидат #A1B2», пока кандидат не

Требование ТЗ 3.5	Реализация
	открыл имя в настройках приватности

Интеграция с реестром ФСП

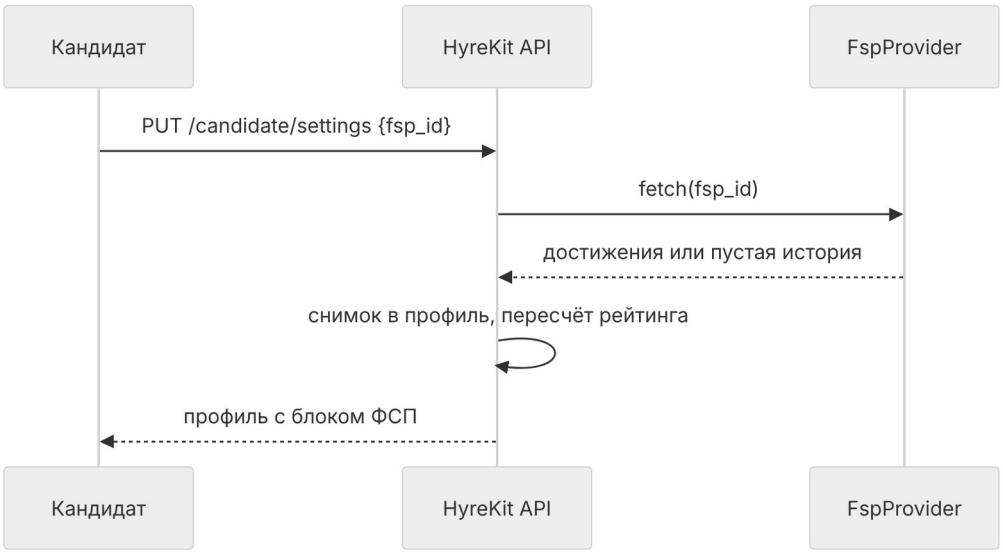
API и структура данных ФСП в хакатоне не предоставлены (ТЗ 4), поэтому интеграция спроектирована через интерфейс и эмулятор.

Что берём из реестра

Поле	Зачем работодателю
Соревнование и его уровень (всероссийский, межрегиональный, региональный)	Масштаб конкуренции
Дисциплина (продуктовое, алгоритмическое программирование, ИБ, робототехника, беспилотные системы)	Связь с профилем вакансии
Место	Сила результата
Год	Актуальность
Роль в команде (капитан, участник)	Лидерский опыт

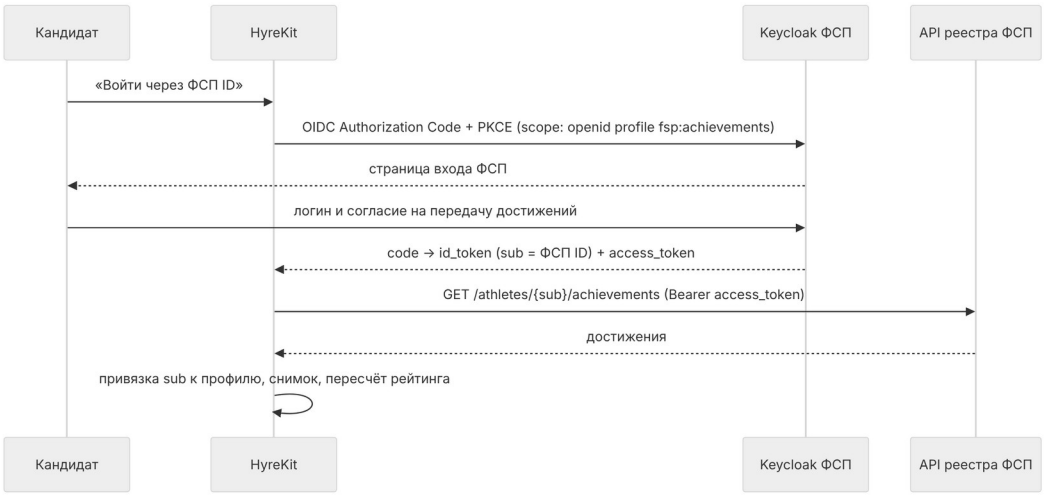
Балл ФСП 0–100 складывается из уровня соревнования, места и давности, а дисциплина, профильная для специализации кандидата, весит вдвое больше (`fsp.provider.summarize`). Работодатель видит список достижений с отметкой «профильное» и итоговый балл. Кандидат без истории получает `has_history: false`: блок показывает «Достижений ФСП пока нет», рейтинг считается по тесту и свежести.

Как устроено в MVP



`MockFspProvider` детерминирован по ФСП ID формата `FSP-NNNN…`: один и тот же ID всегда даёт одну историю, а ID, оканчивающийся на 0, моделирует участника без истории. Ответ сохраняется снимком в профиле, поэтому подборка не зависит от доступности реестра.

Целевая схема с ФСП ID на Keycloak



Замена эмулятора — реализация того же протокола `FspProvider.fetch` поверх HTTP-клиента; остальной код не меняется. Вход через Keycloak добавляется как второй способ аутентификации: HyreKit выпускает свои JWT после проверки `id_token`, поэтому кабинеты и роли не меняются. Обновление достижений — периодически или по вебхуку реестра.

Масштабирование

- API без состояния: горизонтально масштабируется за балансировщиком; лимиты запросов в Redis общие для всех экземпляров.
- Подборка считается в памяти по видимым профилям направления. Для MVP-объёмов (тысячи кандидатов на направление) это миллисекунды; при росте — предварительный отбор в SQL по категории и стеку (JSON-индексы PostgreSQL) и кэш подборки на вакансию.
- Банк заданий — код, а не таблица: новые семейства добавляются без миграций.

Концепция за рамками MVP

Защита от фиктивных вакансий и недобросовестных работодателей

1. **Верификация компании** по ИНН через открытые реестры (ЕГРЮЛ) и подтверждение корпоративного домена почты.
2. **Обязательная вилка зарплаты** уже есть; добавляется проверка правдоподобия: вилка сравнивается с медианой категории, выбросы уходят на модерацию.
3. **Репутация работодателя** по поведению в системе: доля приглашений без ответа на отклики, доля отозванных предложений, жалобы кандидатов. Низкая репутация ограничивает число приглашений в сутки.
4. **Лимиты**: не больше N приглашений в сутки для непроверенной компании; повторное приглашение того же кандидата уже запрещено, пока первое не закрыто.
5. **Сигналы фейка**: одинаковые тексты вакансий у разных компаний, вакансии без откликов работодателя, ссылки на внешние мессенджеры в тексте.

Интеграция с ATS

- **Исходящая**: вебхуки о событиях (`invitation.accepted`, `application.created`) с подписью HMAC; ATS подписывается на них и заводит кандидата.
- **Входящая**: API-ключ компании с правами на создание вакансий и смену статусов откликов, чтобы вакансии из ATS публиковались автоматически.
- **Формат**: карточка кандидата и стандартизированный PDF-профиль уже есть; добавляется экспорт в JSON Resume для импорта в ATS.

5. Запуск и демо

Вариант 1. Docker Compose — одна команда

Нужно: Docker 24+ с Compose v2, 2 ГБ свободной памяти. Работает на Linux, macOS и Windows (WSL2).

```
git clone https://github.com/Fighter90/hyrekit_fsp.git
cd hyrekit_fsp
cp .env.example .env
docker compose up -d --build
```

Что	Адрес
Приложение	http://localhost:3000
Swagger UI	http://localhost:8000/docs
Письма подтверждения (Mailpit)	http://localhost:8025

При первом старте API применяет миграцию и создаёт демо-данные: два аккаунта, компанию с тремя вакансиями и двумя заданиями, 120 синтетических кандидатов по шести направлениям. Повторный старт ничего не дублирует.

Проверка, что всё поднялось:

```
curl -s http://localhost:8000/health        # {"status":"ok",...}
docker compose ps                           # app, db, redis, mailpit,
frontend — healthy / running
```

Полная проверка стенда одной командой — `python scripts/smoke.py` (нужен `httpx` из `requirements.txt`): 28 шагов сквозного сценария через живой API, включая письмо в Mailpit; для удалённого стенда — `--base https://hyrekit.ru --no-mail`.

Остановка: `docker compose down`; с удалением данных — `docker compose down -v`.

Вариант 2. Локально без Docker и PostgreSQL

Нужно: Python 3.11+, Node.js 20+.

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
PATH=$PWD/.venv/bin:$PATH scripts/dev.sh          # API на
http://localhost:8000, база — файл .dev.db (SQLite)

# во втором терминале
cd frontend && npm ci && npm run dev               #
http://localhost:5173, /api проксируется на :8000
```

`scripts/dev.sh` выставляет `DATABASE_URL` на SQLite, применяет миграцию и запускает API с автоперезагрузкой. Демо-данные создаются так же, как в Docker. Ссылка подтверждения почты без SMTP печатается в лог API.

Демо-аккаунты

Роль	Почта	Пароль	Что внутри
Кандидат	`candidate@hyrekit.ru`	`HyreKit2026`	Пустой профиль — путь проходится с нуля
Работодатель	`employer@hyrekit.ru`	`HyreKit2026`	Компания «ТехноПлатформа», 3 вакансии с подборками, 2 коротких задания

Синтетические кандидаты (`talent01@demo.hyrekit.ru` … `talent120@demo.hyrekit.ru`, тот же пароль) прошли тест через симуляцию по той же модели, что и валидация, поэтому их категории и рейтинги согласованы с методикой. Около 65 % привязали ФСП ID; часть ID без истории — так видно, как система показывает кандидата без достижений.

Демо за 5 минут

Сквозной сценарий ТЗ 3.3.

Кандидат (`candidate@hyrekit.ru`):

1. «Профиль» → загрузить PDF-резюме: поля заполнятся автоматически, распознанные подсвечены. Либо заполнить вручную.
2. «Тестирование» → опрос: отрасль, Backend, стаж, стек, предполагаемый грейд Middle.
3. Пройти тест: 10 заданий, таймер, карта вопросов, ответы сохраняются сами.
4. Результат: балл, разбор по темам, присвоенная категория «Backend · Middle» или предложение пересдать уровнем ниже.
5. «Настройки» → дать согласие на публикацию, привязать ФСП ID, например `FSP-123457` (есть история) или `FSP-123450` (истории нет).

Работодатель (`employer@hyrekit.ru`, в другом браузере или окне инкогнито):

6. «Потребности» → «Python backend-разработчик» → подборка: кандидаты по убыванию соответствия, полосы компонентов балла,

причины, недостающий стек. Свежий кандидат из шагов 1–5 появится здесь.

7. Отфильтровать «Только с достижениями ФСП» и сбросить фильтр — исходная подборка не теряется.
8. Вкладка «Тест под вакансию» — темы из стека вакансии и два разных варианта одинаковой сложности.
9. Открыть кандидата → контакты скрыты → «Пригласить»: описание, вилка 250 000 – 320 000 ₽, способ связи.

Кандидат:

10. «Приглашения» → видно компанию и зарплату → «Принять».

Работодатель:

11. «Приглашения и отклики» → статус «принято», у кандидата открылись ФИО и контакты.

Дополнительно: кандидат откликается на опубликованную вакансию в «Вакансиях», работодатель видит отклик и раскрытые контакты; кандидат отвечает на короткое задание в «Челленджах», работодатель видит ответ с оценкой.

Переменные окружения

Переменная	По умолчанию	Назначение
`SECRET_KEY`	— (обязательна)	Подпись JWT; в production слабый ключ останавливает запуск
`DATABASE_URL`	PostgreSQL на localhost	`postgresql+asyncpg://…` или `sqlite+aiosqlite:///…`
`REDIS_URL`	`redis://localhost:6379/0`	Лимиты запросов
`ENVIRONMENT`	`development`	`production` включает строгие проверки
`CORS_ORIGINS`	localhost	Разрешённые адреса фронтенда
`GIGACHAT_CREDENTIALS`	пусто	Ключ GigaChat; без него разбор резюме и оценка заданий идут по правилам
`GIGACHAT_SCOPE`, `GIGACHAT_MODEL`	`GIGACHAT_API_PERS`, `GigaChat`	Параметры GigaChat
`SEED_DEMO`	`true`	Демо-аккаунты и синтетический банк при старте

Переменная	По умолчанию	Назначение
`PUBLIC_URL`	`http://localhost:3000`	Адрес фронтенда в ссылках писем
`DOCS_ENABLED`	`true`	Swagger UI и `/openapi.json`
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `SMTP_TLS`, `MAIL_FROM`	пусто	Почта; пустой `SMTP_HOST` — ссылка в лог

Сборка и проверки

```
ruff check src tests scripts && ruff format --check src tests scripts
# линтер и формат
pytest --cov=app
# тесты бэкенда с покрытием
PYTHONPATH=src python scripts/validate.py
# валидация методики → docs/validation/results.md
cd frontend && npm run lint && npm run build && npm test
# фронтенд
```

Развёртывание в Yandex Cloud

Целевая площадка — https://hyrekit.ru. Схема: одна ВМ Compute Cloud (2 vCPU, 4 ГБ) с Docker Compose из этого репозитория, Caddy или nginx с сертификатом Let's Encrypt перед контейнером `frontend`, ежедневный снимок диска. Пошаговая инструкция — docs/runbook.md, файлы площадки — `deploy/yc/`.

6. Технологии и сервисы

Весь стек — открытое ПО под разрешительными лицензиями (ТЗ 3.5). Версии бэкенда закреплены в `requirements.txt` и `requirements-dev.txt`, фронтенда — в `frontend/package-lock.json`.

Бэкенд (Python 3.11+)

Библиотека	Версия	Лицензия	Зачем
FastAPI	0.115.6	MIT	HTTP API и генерация OpenAPI
Uvicorn	0.34.0	BSD-3	ASGI-сервер
Pydantic / pydantic-settings	2.10.5 / 2.7.1	MIT	Схемы, валидация, конфигурация из окружения
SQLAlchemy	2.0.37	MIT	ORM

Библиотека	Версия	Лицензия	Зачем
(asyncio)			
asyncpg	0.30.0	Apache-2.0	Драйвер PostgreSQL
aiosqlite	0.20.0	MIT	SQLite для локального запуска и тестов
Alembic	1.14.1	MIT	Миграции
redis	5.2.1	MIT	Клиент Redis для лимитов
slowapi	0.1.9	MIT	Ограничение частоты запросов
PyJWT	2.10.1	MIT	Токены доступа
bcrypt	4.2.1	Apache-2.0	Хеширование паролей
email-validator	2.2.0	Unlicense	Проверка адресов
python-multipart	0.0.20	Apache-2.0	Загрузка файлов
PyMuPDF	1.25.3	AGPL-3.0	Извлечение текста из PDF-резюме
python-docx	1.1.2	MIT	Извлечение текста из DOCX-резюме
reportlab	5.0.1	BSD	Генерация PDF-профиля
gigachat	0.1.40	MIT	Клиент GigaChat
httpx	0.28.1	BSD-3	HTTP-клиент
tenacity	9.0.0	Apache-2.0	Повторы внешних вызовов
structlog	24.4.0	MIT / Apache-2.0	Логи

Разработка: pytest 8.3.4, pytest-asyncio 0.24.0, pytest-cov 6.0.0, ruff 0.8.6, mypy 1.14.1, bandit 1.8.3, pip-audit 2.7.3.

Фронтенд (Node.js 20+)

Пакет	Версия	Лицензия	Зачем
React / React DOM	19.2.4	MIT	Интерфейс

Пакет	Версия	Лицензия	Зачем
React Router	6.30.3	MIT	Маршруты кабинетов
lucide-react	0.576.0	ISC	Иконки
TypeScript	5.8.3	Apache-2.0	Типы
Vite	6.4.1	MIT	Сборка и dev-сервер
Vitest, Testing Library	4.0.18, 16.3.2	MIT	Тесты
ESLint	9.39.3	MIT	Линтер

Инфраструктура

Компонент	Версия	Зачем
PostgreSQL	16 (alpine)	Основное хранилище
Redis	7 (alpine)	Лимиты запросов
nginx	1.27 (alpine)	Статика фронтенда и прокси `/api`
Mailpit	1.21	Перехват писем в локальном стенде
Docker Compose	v2	Запуск всех компонентов одной командой

Шрифты и бренд

Ресурс	Лицензия	Где
JetBrains Mono — основной шрифт ФСП	SIL OFL 1.1	`frontend/public/fonts/`, `src/app/assets/fonts/` (PDF-профиль)
TDAText — акцентный шрифт ФСП	SIL OFL 1.1	`frontend/public/fonts/`
Логотип и паттерн ФСП	Брендбук ФСП, предоставлен организаторами	`frontend/public/brand/`

GigaChat

Используется в двух местах и нигде не принимает решений о грейде или подборке:

Где	Что делает	Без ключа
Разбор резюме (`candidates.resume_parser`)	Извлекает из текста ФИО, контакты, стек, роли, стаж, софт-скиллы в JSON; результат сливается с правилами	Регулярные выражения и словари технологий
Оценка коротких заданий (`challenges.service`)	Оценивает ответ по рубрике и пишет отзыв	Эвристика по полноте и ключевым словам задачи

Задания теста модель не генерирует и не проверяет: это гарантирует сопоставимую сложность и воспроизводимую оценку (см. раздел 3). Ключ задаётся `GIGACHAT_CREDENTIALS`; клиент — `src/app/llm/gigachat.py`.

Полный список сторонних компонентов и их лицензий — THIRD_PARTY_NOTICES.md.

7. Команда и процесс

Команда и трек

Параметр	Значение
Команда	Команда 1
Трек	«Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника ФСП»
Хакатон	Хакатон Федерации спортивного программирования, 2026

Участник	Роль
Яков Дятлов (@iakov_is)	product manager
Ирина Живова (@IrinaZhivova)	product manager
Сергей Емельянов (@sergey_in_job)	backend developer
Алиса Живова (@AlisaZhivova)	аналитик

Как велась разработка

Источник истины — задачи Linear в проекте «HyreKit — платформа подбора ИТ-специалистов (ФСП)», ключи `FSP-N`. Эпики повторяют критерии оценки ТЗ: механика подбора (35 %), механика тестирования и

категоризация (25 %), интерфейс (10 %), дополнительный функционал (7 %), валидация, документация и сдача.

Spec-Driven Development. Путь одной задачи:

1. Задача в Linear: контекст, что сделать, правила и числа со ссылками на пункты ТЗ, критерии приёмки.
2. Спецификация в `docs/specs/fsp-N-*.md` по шаблону `_template.md`: цель, сценарии, правила, критерии приёмки в формате EARS, план тестов.
3. Контракт API в `docs/spec/api.md` — до кода; фронтенд и бэкенд пишутся по нему параллельно.
4. Код и тесты на каждый критерий приёмки.
5. Архитектурные решения с альтернативами — в `docs/decisions/`.

Подробно — docs/process.md.

Качество

Что	Как проверяется
Линтер и формат бэкенда	`ruff check`, `ruff format --check`
Тесты бэкенда	pytest, 183 теста, покрытие кода 81 % при пороге 70 %
Сквозной сценарий	`tests/test_flow.py`: регистрация → опрос → тест → категория → подборка → приглашение → раскрытие контактов; провал теста без понижения грейда; тест под вакансию
Методика	`tests/test_method/`: каждое семейство заданий на всех уровнях генерирует проверяемое задание; монотонность оценки θ; правила грейда; подбор; ФСП
Валидация	`scripts/validate.py` в CI: равная сложность, утечка, надёжность, дискриминативность, точность грейда, precision@10
Фронтенд	ESLint, `tsc`, сборка Vite, тесты Vitest

CI — GitHub Actions, `.github/workflows/ci.yml`: два задания, бэкенд и фронтенд, на каждый push и pull request.

Критерии оценки и где они закрыты

Критерий ТЗ	Вес	Что в продукте	Где смотреть
Новая механика	35 %	Приглашение с	раздел 3, экран

Критерий ТЗ	Вес	Что в продукте	Где смотреть
подбора		вилкой от работодателя, подборка с объяснимостью, раскрытие контактов по согласию; P@10 0.87–0.96	подборки
Тестирование, валидация, категоризация	25 %	Параметрические семейства, равная сложность, оценка θ, грейд без понижения, процедура валидации	раздел 3, результаты
Удобство интерфейса	10 %	Два кабинета в фирменном стиле ФСП, адаптивная вёрстка	скриншоты
Техническая проработка	7 %	Модульный монолит, OpenAPI, Docker Compose, CI	раздел 4
Полнота и документация	6 %	Все обязательные пункты ТЗ 2.1, профиль без ФСП, этот комплект документов	этот раздел
Дополнительный функционал	7 %	Публичные вакансии и отклики, короткие задания, концепция защиты от фиктивных вакансий и ATS, мобильная версия	раздел 2, раздел 4